



Traffic Sign Classification

U Kang
Seoul National University



In This Lecture

- German Traffic Sign Benchmarks
- Multi-class classification from traffic sign images



Outline

- ➡ ☐ Introduction
- ☐ Dataset
- ☐ Preprocessing Codes



About the Task

- The German Traffic Sign Benchmark is a multi-class classification challenge dataset
 - ❑ Link: <https://benchmark.ini.rub.de/index.html>
 - ❑ Number of classes: 43
 - ❑ Imbalanced class distribution



Problem Definition

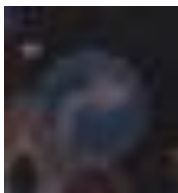
- **Given:** German Traffic Sign data
- **Classify:** test data into the correct categories
 - E.g.



class 0
“Speed limit
(20km/h)”



class 4
“Speed limit
(70km/h)”



class 33
“Turn right
ahead”



class 37
“Go straight or
left”



Outline

☒ Introduction

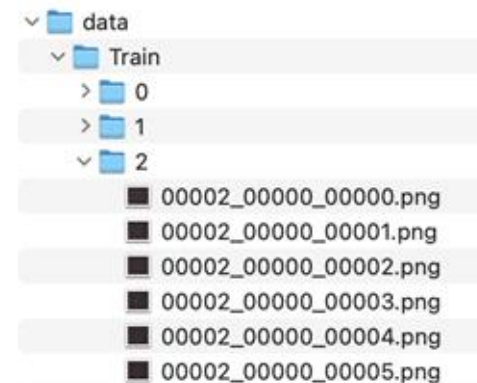
 ☐ **Dataset**

☐ Preprocessing Codes



Training Dataset (1)

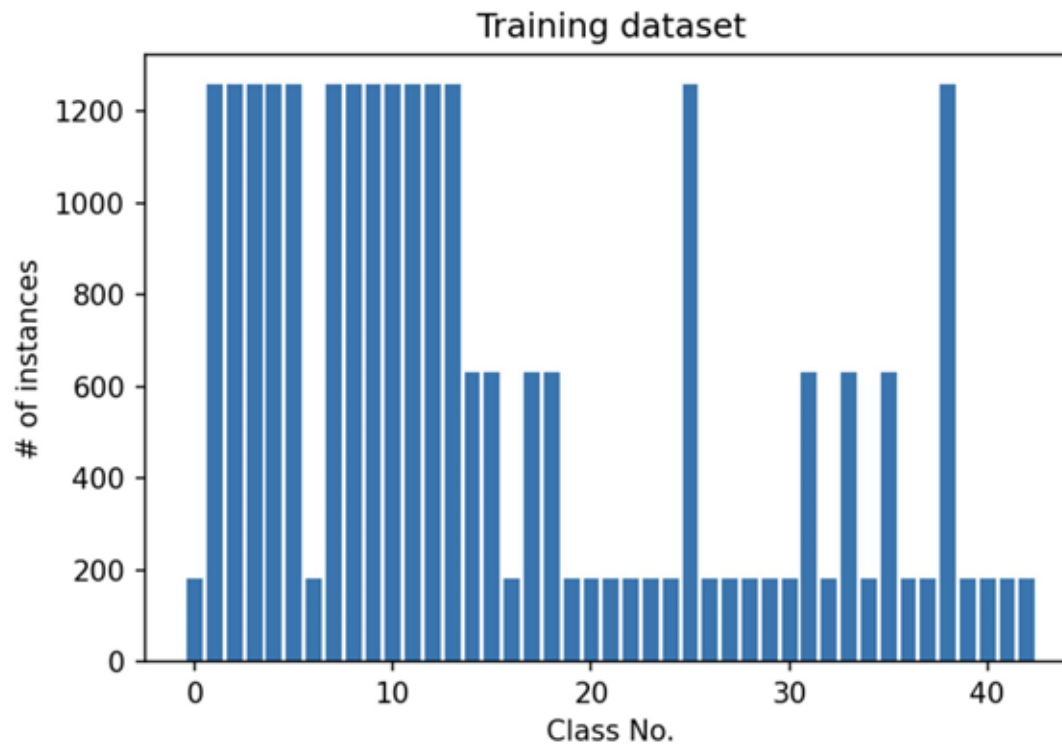
- Approximately 30,000 images
- Image sizes vary between 15x15 to 250x250 pixels
 - Images are not necessarily squared
- The training set archive is structures as follows:
 - One directory per class
 - Training images are grouped by tracks
 - Each track contains 30 images of a single physical traffic sign
 - Filename format
 - {class id}_{track id}_{ith image for the track}.png





Training Dataset (2)

- Label distribution of training data





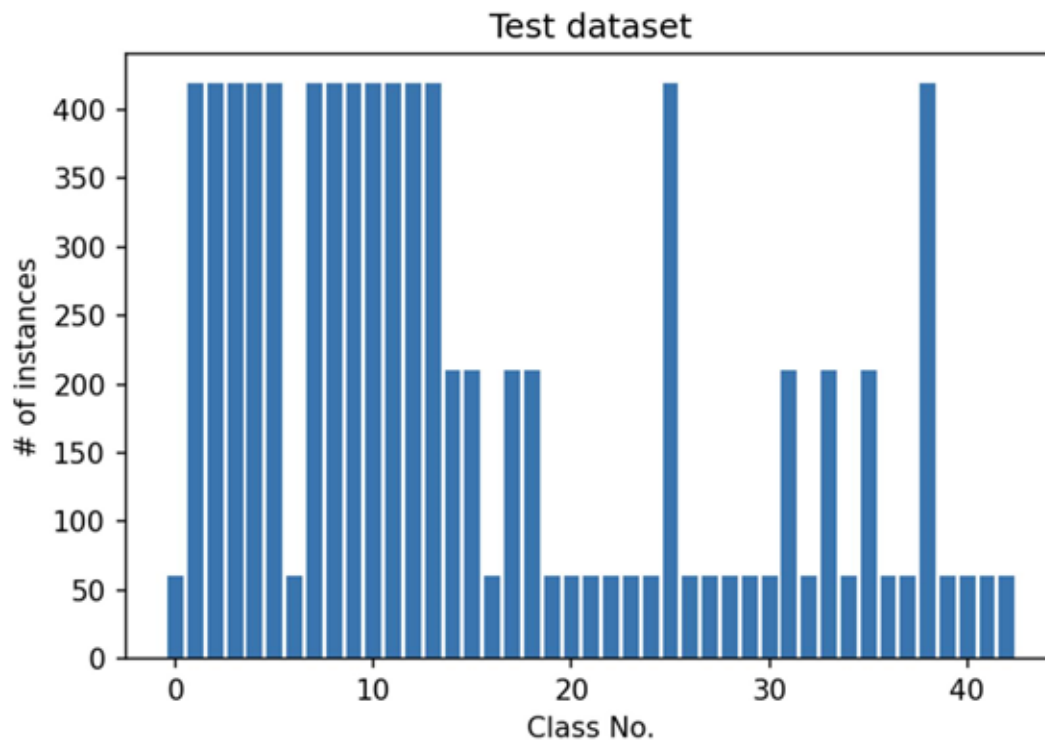
Test Dataset (1)

- Approximately 10,000 images
- Image sizes vary between 15x15 to 250x250 pixels
 - Images are not necessarily squared



Test Dataset (2)

- Label distribution of test data





Outline

☒ Introduction

☒ Data

 ☐ **Preprocessing Codes**



Import Libraries

- Import the libraries such as os, numpy, cv2, and torch

```
import os
from os import listdir
from os.path import isfile, join

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import cv2
from PIL import Image
from sklearn.metrics import f1_score
from sklearn.utils.class_weight import compute_class_weight

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
```



Loading the Dataset (1)

- Extracts zip files to their corresponding folder

```
def safe_unzip(zip_filename, extract_path):  
  
    if not os.path.exists(zip_filename):  
        print(f"오류: 압축 파일 '{zip_filename}'을 찾을 수 없습니다.")  
        return  
  
    if os.path.isdir(extract_path):  
        print(f"폴더 '{extract_path}'가 이미 존재합니다. 압축 해제를 건너뛴니다.")  
        return  
  
    print(f"폴더 '{extract_path}'가 없으므로 '{zip_filename}' 압축을 해제합니다...")  
  
    try:  
        with zipfile.ZipFile(zip_filename, 'r') as zip_ref:  
            zip_ref.extractall(extract_path)  
        print(f"압축 해제 완료: '{extract_path}' 폴더가 생성되었습니다.")  
  
    except Exception as e:  
        print(f"압축 해제 중 오류 발생: {e}")  
  
safe_unzip('data.zip', './data/')
```

폴더 './data/'가 없으므로 'data.zip' 압축을 해제합니다...
압축 해제 완료: './data/' 폴더가 생성되었습니다.



Loading the Dataset (2)

- Read train data and put images and labels into numpy arrays

```
data = []
labels = []
height = 30
width = 30
channels = 3
num_classes = 43

for i in range(num_classes):
    path = f'./data/Train/{i}/'
    Class = os.listdir(path)
    for a in Class:
        image = cv2.imread(path + a)
        image_from_array = Image.fromarray(image, 'RGB')
        resized_image = image_from_array.resize((height, width))
        data.append(np.array(resized_image))
        labels.append(i)

data = np.array(data)
labels = np.array(labels)
```



Loading the Dataset (3)

■ Shuffle the train data

```
s = np.arange(data.shape[0])  
np.random.shuffle(s)  
  
data = data[s]  
labels = labels[s]
```



Loading the Dataset (4)

- Converts the data type to float32 and scales the pixel values from 0 to 1
- Splits the data into train and validation sets at an 8:2 ratio

```
data = data.astype('float32') / 255

X_train = data[(int)(0.2 * len(labels)):]
X_val = data[: (int)(0.2 * len(labels))]

y_train = labels[(int)(0.2 * len(labels)):]
y_val = labels[: (int)(0.2 * len(labels))]

print(f"Train X: {X_train.shape}, Train y: {y_train.shape}")
print(f"Val X: {X_val.shape}, Val y: {y_val.shape}")

Train X: (20808, 30, 30, 3), Train y: (20808,)
Val X: (5202, 30, 30, 3), Val y: (5202,)
```



Test

■ Test measure

□ Macro F1 score

- The arithmetic mean (unweighted mean) of all the per-class F1 scores

$$\text{F1 Score} = \frac{\text{TP}}{\text{TP} + \frac{1}{2}(\text{FP} + \text{FN})}$$

1

No	Actual	Predicted	Match
1	Airplane	Airplane	✓
2	Car	Boat	✗
3	Car	Car	✓
4	Car	Car	✓
5	Car	Boat	✗
6	Airplane	Boat	✗
7	Boat	Boat	✓
8	Car	Airplane	✗
9	Airplane	Airplane	✓
10	Car	Car	✓

2

Label	True Positive (TP)	False Positive (FP)	False Negative (FN)	Precision	Recall	F1 Score
Airplane	2	1	1	0.67	0.67	$2 * (0.67 * 0.67) / (0.67 + 0.67) = \mathbf{0.67}$
Boat	1	3	0	0.25	1.00	$2 * (0.25 * 1.00) / (0.25 + 1.00) = \mathbf{0.40}$
Car	3	0	3	1.00	0.50	$2 * (1.00 * 0.50) / (1.00 + 0.50) = \mathbf{0.67}$

3

Label	Per-Class F1 Score	Macro-Averaged F1 Score
Airplane	0.67	$\frac{0.67 + 0.40 + 0.67}{3} = \mathbf{0.58}$
Boat	0.40	
Car	0.67	



Test

- Save results to csv file.
 - ❑ Save path: './result.csv'
 - ❑ 1st column: test file name
 - ❑ 2nd column: predicted class
 - Separator: comma

1	00000.png,12
2	00001.png,11
3	00002.png,33
4	00003.png,33
5	00004.png,11
6	00005.png,3
7	00006.png,8
8	00007.png,42
9	00008.png,25
10	00009.png,35



Questions?